ENTERPRISE REAL-TIME DATA ENGINEERING

Session S10: Spark Structured Streaming & Event-Time Processing

Unbounded Table Model, Event Time vs Processing Time, Watermarks & Checkpoint WAL

Course Leadership & Resources

Lead Instructor:
Mohamed Fouad Fakhruldeen (م/ محمد فؤاد فخر الدين)

Personal Contact & Website:
https://fakhr.me

Course Materials & Slide Portal:
https://fakhr.me/KafkaSpark

Student Labs & Code Repository:
https://github.com/fakhruldeen/KafkaSpark

Core Modules & Architectural Pillars

✔️ **The Unbounded Table Model & Modes:** Unifying batch DataFrames with continuous streams; Micro-batch vs Continuous mode

✔️ **Event Time vs Processing Time:** The train delay scenario, reconciling out-of-order data & mathematical watermarking

✔️ **Stateful Stream Joins & lastProgress:** Stream-static vs stream-stream joins, dual watermarks, and lastProgress JSON inspection

✔️ **Checkpointing & WAL Forensics:** offsets/, commits/, and state/ directories for exactly-once recovery

MODULE ROADMAP & CURRICULUM FOCUS

Engineering Agenda & Learning Outcomes

Structured Streaming Engine

Continuous Processing:

• Micro-batch execution model and StreamingQueryManager.

• Unifying batch DataFrame APIs with real-time streaming sources.

Checkpointing & WAL

Deterministic State Recovery:

• Write-Ahead Log (WAL) records offset ranges before task execution.

• State store delta commits guaranteeing zero gaps and zero duplicate writes.

Event-Time Watermarks

State Bounding & Late Data:

• Watermark = max(event_time) - delay_threshold.

• Safe eviction of expired window state, preventing JVM OutOfMemory crashes.

HISTORICAL ORIGIN STORY

The Time Crisis: The Train Delay & Processing Time Accounting Failure

The Time Crisis: The Train Delay & Processing Time Accounting Failure

A major retail banking application processed credit card transactions using Processing Time (the system clock timestamp of the server receiving the event). A commuter on a train initiated a $1,000 transaction at 10:00 AM while passing through a rural tunnel. Due to network failure, the mobile phone buffered the transaction and did not successfully transmit it until 12:00 PM when the train reached the central terminal. The bank's real-time system recorded the sale at 12:00 PM, creating false revenue spikes at noon and an accounting audit deficit for the morning shift.

The Flaw of Processing Time in Financial Systems

• Time Distortion: Processing Time reflects network transmission delays and server load rather than when the real-world business event occurred.
• Non-Reproducibility: Replaying yesterday's events today yields completely different windowed results because current wall-clock time is used.
• Unbounded State Store Growth: Without a mathematical horizon to close historical windows, the stream engine must hold state for eternity, crashing with OOM.

The Event Time & Watermarking Revolution

• Event Time Invariant: Windows and aggregations are evaluated strictly using the timestamp embedded inside the event data (`event_time`).
• Mathematical Watermarking: `Watermark = max(event_time) - delay_threshold`. Defines the exact point in time when Spark considers all past data finalized.
• State Store Garbage Collection: Once the watermark advances past a window's end time, Spark purges the window from memory, bounding state indefinitely.

Core Architecture & Kernel Primitives

Unbounded Table

STREAMING ABSTRACTION

Formal Definition:
The conceptual model where a live streaming source is treated as an append-only table that grows indefinitely with each incoming event.

Kernel / Technical Mechanics:
Every micro-batch executes an incremental query against new rows appended to the unbounded table, updating results incrementally.

Critical Failure Mode:
Attempting to call non-streaming operations (e.g. `df.show()`, multiple non-watermarked aggregations) throws AnalysisException.

Event Time

TEMPORAL ATTRIBUTE

Formal Definition:
The exact time at which the real-world event occurred on the client device or sensor, embedded directly inside the message payload.

Kernel / Technical Mechanics:
Extracted via `F.to_timestamp(F.col('event_time'))`. Allows accurate aggregation even if messages arrive out of chronological order.

Critical Failure Mode:
Clock skew on client devices emitting events from years in the past or future, distorting watermark calculations.

Watermark Delay

STATE EVICTION BOUNDARY

Formal Definition:
The threshold specifying how long Spark should wait for late data before closing a window and dropping subsequent late events.

Kernel / Technical Mechanics:
Applied via `.withWatermark('event_ts', '10 minutes')`. Watermark = max(event_ts seen) - 10 minutes. State is evicted once watermark passes window end.

Critical Failure Mode:
Setting delay too short drops valid delayed transactions; setting delay too long inflates state store memory footprint.

Advanced Invariants & Protocol Guarantees

Output Modes

EGRESS CONTRACT

Formal Definition:
Determines what data is written to the streaming sink on each micro-batch trigger: Append, Update, or Complete mode.

Kernel / Technical Mechanics:
Append: Writes only finalized, closed rows. Update: Writes only rows modified in the current batch. Complete: Re-writes entire aggregated table.

Critical Failure Mode:
Mode incompatibility: Using Append mode on streaming aggregations without a watermark throws AnalysisException.

Write-Ahead Log (WAL)

DURABILITY PRIMITIVE

Formal Definition:
The persistent transaction log maintained inside the checkpoint directory that guarantees exactly-once recovery across restarts.

Kernel / Technical Mechanics:
Directory layout: `offsets/` (batch start/end offsets), `commits/` (completed batches), and `state/` (serialized RocksDB/HDFS delta snapshots).

Critical Failure Mode:
Deleting the checkpoint directory to 'fix' a bug causes the engine to re-read from earliest offsets, duplicating months of data.

Watermarked dropDuplicates

EXACTLY-ONCE DEDUPLICATION

Formal Definition:
Stateful deduplication of streaming events across time bounded by an explicit event-time watermark.

Kernel / Technical Mechanics:
Applied via `.withWatermark('event_ts', '15m').dropDuplicates(['event_id'])`. Tracks unique IDs in StateStore only until watermark passes, preventing memory leaks.

Critical Failure Mode:
Calling dropDuplicates without a watermark keeps keys in memory indefinitely, eventually crashing executors with OutOfMemoryError.

VISUAL ARCHITECTURE DIAGRAM

Visual Data Flow: Structured Streaming Micro-Batch & Watermark Lifecycle



Event-Time Watermark Progression Invariant

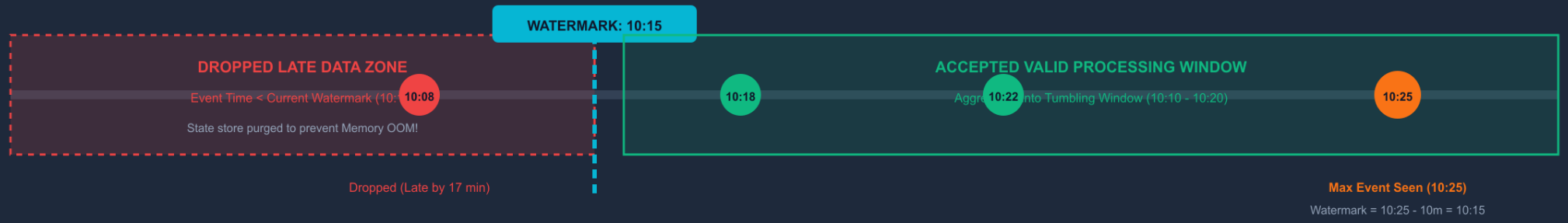
The Kafka source streams records. Spark evaluates max(event_ts), updates the Watermark, aggregates rows within 1-minute tumbling windows, drops late records arriving behind the watermark, and flushes output rows to Kafka with atomic offset commits.

DEEP ARCHITECTURE ANALYSIS

Execution Internals & Trade-off Matrix

EVENT-TIME STREAMING: Watermarking, Late Data & Checkpoint Directory

1. EVENT-TIME WATERMARK HORIZON: withWatermark("event_time", "10 minutes")



2. END-TO-END EXACTLY-ONCE (EOS): WRITE-AHEAD LOG & CHECKPOINT DIRECTORY

/checkpoints/offsets/

- Commits exact Kafka partition offsets
- Written to disk BEFORE executing micro-batch

✓ Prevents reprocessing skipped offsets

/checkpoints/commits/

- Idempotent batch completion record
- Identifies batch ID 0, 1, 2... as successful

✓ Guarantees idempotent output writes

/checkpoints/state/

- RocksDB state store delta files
- Backs up running window aggregations

✓ Zero state loss upon executor crash!

THEORETICAL FOUNDATIONS

Mathematical Proofs & Distributed Invariants

Event-Time vs Processing-Time

Temporal Semantics Law:

Event time is embedded in data records (T_event); processing time is the executor clock.

Windowed streaming aggregations MUST bucket by event time to handle out-of-order arrivals correctly.

Watermark State Bounding

State Bounding Invariant:

Watermark = max(T_event) - Delta_delay. State windows older than the watermark are evicted from memory.

Guarantees state store memory stays bounded indefinitely, preventing OutOfMemory crashes.

Micro-Batch WAL & Recovery

Exactly-Once Replay Law:

Driver commits source partition offset ranges to Write-Ahead Log (WAL) before tasks execute.

Crash restarts re-read WAL to execute exact micro-batch slices idempotently without gaps.

VISUAL TOPOLOGY

Cluster Mesh & Component Architecture

Kafka Micro-Batch Source

spark-sql-kafka-0-10

- Driver queries Kafka partition end offsets.
- Calculates micro-batch offset slices.
- Executors pull assigned byte ranges directly.
- Zero consumer group rebalance interference.

WAL Checkpoint Directory

data/checkpoints/s10

- offsets/: Atomic micro-batch start/end offsets.
- commits/: Acknowledged micro-batch completions.
- state/: HDFS/file-backed state store snapshots.
- metadata: Query UUID and schema contracts.

Streaming Sink Target

Console / Storage Sink

- Materializes aggregated window metrics.
- Trigger(processingTime='5 seconds').
- Idempotent output commit per batch ID.
- Sub-second stream processing latency.

PRODUCTION IMPLEMENTATION

Enterprise Configuration & Code Blueprint

Structured Streaming Ingestion Code

```
stream_df = spark.readStream.format('kafka') \
  .option('kafka.bootstrap.servers', 'localhost:9092') \
  .option('subscribe', 'orders.events.v1') \
  .option('startingOffsets', 'earliest') \
  .option('maxOffsetsPerTrigger', 500) \
  .load()

parsed_df = stream_df.select(
  from_json(col('value').cast('string'), order_schema).alias('data')
).select('data.*')
```

Windowed Aggregation & Checkpointing

```
# Watermarked Aggregation with Checkpointing:
query = parsed_df \
  .withWatermark('timestamp', '10 minutes') \
  .groupBy(window('timestamp', '5 minutes'), 'store_id') \
  .agg(sum('amount').alias('revenue')) \
  .writeStream \
  .outputMode('update') \
  .format('console') \
  .option('checkpointLocation', 'data/checkpoints/s10') \
  .start()
```

ENTERPRISE CASE STUDY

Nile Retail Streaming Dashboard Freeze

Nile Retail Streaming Dashboard Freeze

During a flash sale, Nile Retail's real-time streaming revenue dashboard froze after 3 hours. Driver heap memory grew monotonically until the JVM collapsed with java.lang.OutOfMemoryError: Java heap space.

Unbounded State Store Leaks

- Streaming aggregation lacked watermarking; every historical store key was kept in RAM.
- State store retained millions of expired transactions from completed days.
- Lack of maxOffsetsPerTrigger allowed traffic surges to crash the driver.

Event-Time Watermarking & Rate Bounding

- Configured .withWatermark('timestamp', '15 minutes') to evict expired state.
- Enforced maxOffsetsPerTrigger=500 to protect executor memory during traffic surges.
- Capped state memory at a stable 350MB, achieving uninterrupted 24/7 streaming uptime.

HANDS-ON LAB EXECUTION

Step-by-Step Terminal Lab Walkthrough

Step 1: Launch Streaming Job

Launch Streaming Pipeline:

```
python -m python.s10_stream \
  --checkpoint data/checkpoints/s10 \
  --max-per-trigger 500
```

Monitor Micro-Batches:
Verify micro-batch progression and trigger rate.

Step 2: Inspect Checkpoint WAL

Inspect WAL Directory:

```
ls -la data/checkpoints/s10/offsets/
```

Verify Offsets:
Examine atomic JSON micro-batch offset commits.

Step 3: Run Verification Suite

Execute Automated Suite:

```
./delivery_materials/
S10_Spark_Structured_Streaming/Trainer/
verify_s10.sh
```

Acceptance Target:
100% pass mark verifying streaming query execution, state bounding, and WAL recovery.

Disaster Recovery Drill: Fault Injection & Self-Healing

Chaos Strike: Mid-Batch Driver Kill

1. Force Kill Spark Streaming Process:
kill -9 <spark_streaming_pid>

2. Mid-Batch Interruption:
- Process killed while writing micro-batch 4.
- Partial output written to storage sink.
- offsets/ contains micro-batch 4, but commits/ lacks batch 4.

3. Restart Streaming Job:
Restart pointing to identical checkpoint directory.

WAL Replay & Exactly-Once Proof

1. Deterministic Replay:
Driver reads offsets/4, re-queries Kafka for exact offset slice, and re-executes.

2. Zero Duplicate Commits:
Idempotent sink overwrites partial batch 4 output with complete committed batch.

3. Zero Record Gaps:
Streaming resumes micro-batch 5 with zero lost records and zero gaps.

PRODUCTION TRAPS & PITFALLS

Critical Anti-Patterns & Operational Lessons

Production Traps: Structured Streaming

Trap 1: Missing Watermark on dropDuplicates()
Calling `dropDuplicates(['event_id'])` without a watermark forces Spark to store EVERY event_id ever seen in memory forever! The State Store grows monotonically until OOM crash.

Trap 2: Changing Schema on an Existing Checkpoint
Modifying column data types causes `StateSchemaCompatibilityChecker` to abort on restart. Streaming schemas require backwards compatibility.

Diagnostic Exit Questions

Q1: Why does Append Mode require a Watermark for windowed aggregates?
A: Because in Append mode, Spark can only output a row when the window is completely finalized and closed; without a watermark, a window never closes!

Q2: What happens to an event whose event_ts < current Watermark?
A: It is identified as late data and permanently dropped from aggregation state.

Q3: Can a checkpoint directory be safely shared between 2 queries?
A: NEVER! Sharing checkpoint directories corrupts the WAL and crashes the engine.

Senior Real-Time Data Engineer Technical Interview Mastery

Senior Technical Interview Scenarios

Q1: 'How does Spark Structured Streaming achieve end-to-end exactly-once semantics?'
A: Replayable source (Kafka offsets) + Deterministic micro-batch DAG + Write-Ahead Log Checkpointing + Idempotent sink output.

Q2: 'What happens to records arriving with timestamps older than the watermark?'
A: They are dropped from stateful aggregations because their time window has already been finalized and evicted from memory.

Core Architectural Competencies

- **Streaming Engine Internals:** Micro-batch execution, trigger intervals, and continuous processing mode.
- **State Store Management:** HDFS-backed state stores, RocksDB state provider, and memory profiling.
- **Fault Tolerance Protocols:** Write-Ahead Log (WAL) structure, commit metadata, and crash replay.

CONCLUSION & NEXT STEPS

Session S10 Key Takeaways & Course Progression

Core Invariants Mastered

✔️ **Micro-Batch Architecture:** Deterministic offset pulls from Kafka brokers.

✔️ **WAL Checkpointing & Watermarks:** Bounded state stores and crash-proof recovery.

Cluster Verified

✔️ **Streaming Checkpoints:** Validated offsets and commits in checkpoint dir.

✔️ **S10 Suite Passed:** 100% test automation pass across streaming gates.

Course Progression

Curriculum Continuity:

Now we unite all layers into an end-to-end enterprise lakehouse!

Next: S11 Kafka & Spark Pipeline Integration & Dual-Branch DLT!

Session S10 Hands-On Lab & Verification Gate

⚡ Hands-On Lab S10: Structured Streaming & Fraud

1. Execute Real-Time Streaming Ingestion:
cd 00_Lab_Environment && source .venv/bin/activate
python3 python/s10_stream.py

2. Launch Real-Time Fraud Alert Pipeline:
python3 python/s10_fraud_alert.py

3. Run Automated Unit Tests:
pytest tests/test_s10_fraud_alert.py -v

4. Automated Lab Verification Gate:
./verify_s10.sh # Validates micro-batch streaming & fraud traps

🎯 Verification & Quality Gate (S10)

1. Automated Verification Script:
./verify_s10.sh

2. Core Acceptance Invariants:
- Structured Streaming micro-batch latency <= 500ms.
- Watermark correctly purges expired aggregation state.
- Checkpointing guarantees exactly-once stream-to-sink delivery.
- Rapid velocity transactions trigger immediate fraud alerts.

3. Stream Progress Observation:
Monitor streamingQuery.lastProgress metrics in real time.

4. Day 4 Streaming Milestone:
End-to-end Structured Streaming from Kafka with stateful watermarks verified!